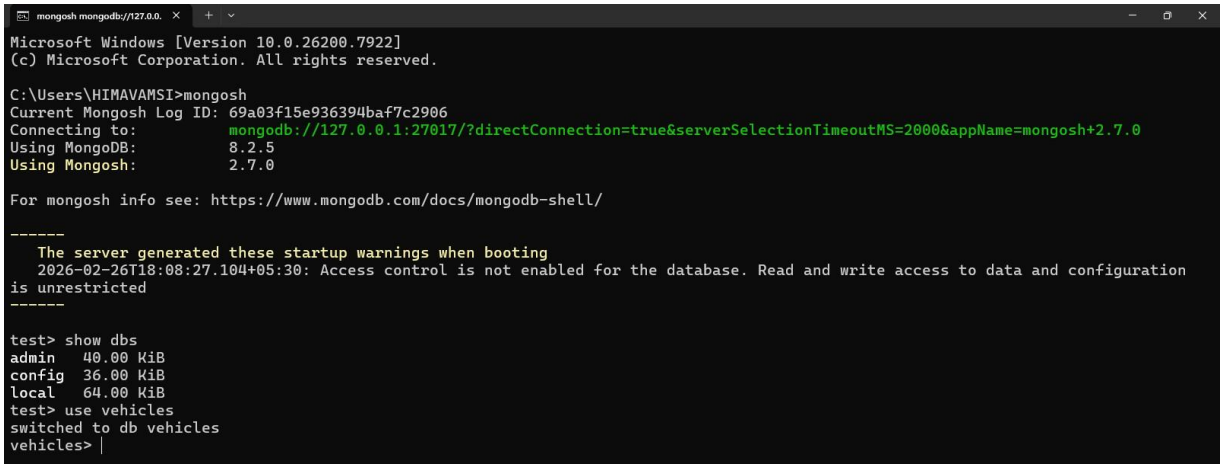
Branch/ Class	**: B.Tech**	**Date**	**: 26/02/2026**
Faculty Name	**: Prof.S.Gopikrishnan**	**School**	**: SCOPE**
Student name	**: K S Saravanesh**	**Reg. no.**	**: 23BCE8183**

1. Use MongoDB to implement the following DB operations

1. Create a database called 'vehicles' and *write* a MongoDB query to select database as "vehicles".

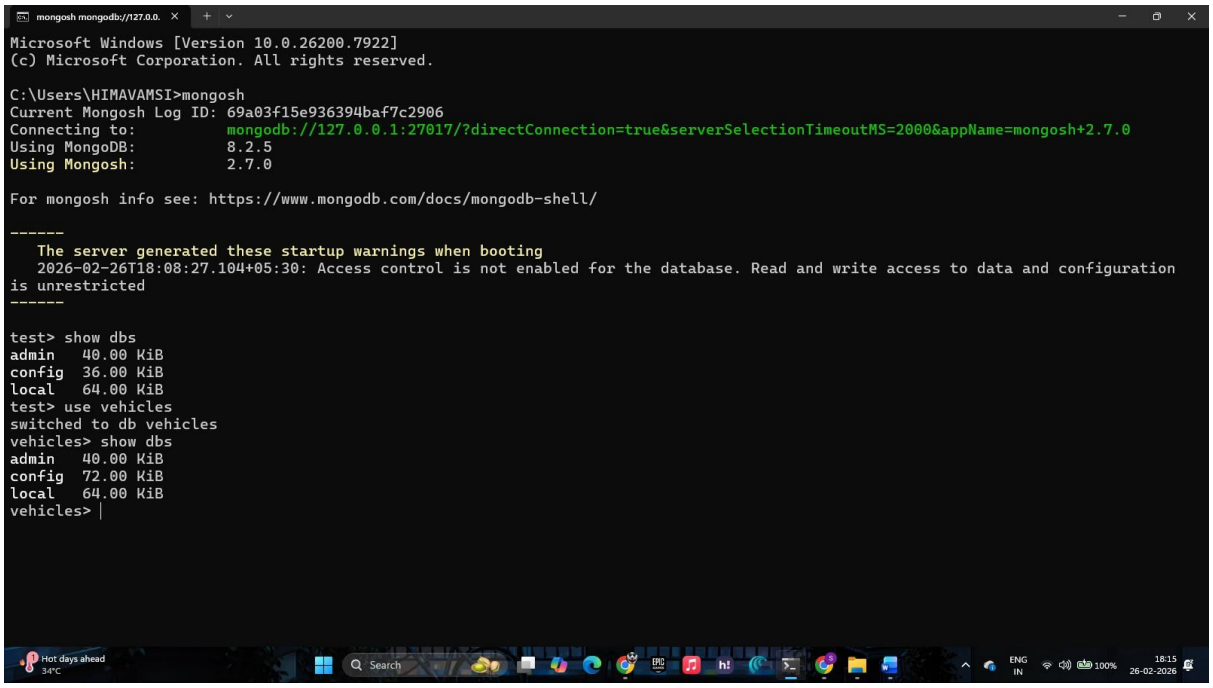
Output:



```
mongosh mongodb://127.0.0.
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HIMAVAMSI>mongosh
Current Mongosh Log ID: 69a03f15e936394baf7c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-02-26T18:08:27.104+05:30: Access control is not enabled for the database. Read and write access to data and configuration
is unrestricted
------

test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   64.00 KiB
test> use vehicles
switched to db vehicles
vehicles>
```

2. Write a MongoDB query to display all the databases.

Output:



```
mongosh mongodb://127.0.0.
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HIMAVAMSI>mongosh
Current Mongosh Log ID: 69a03f15e936394baf7c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-02-26T18:08:27.104+05:30: Access control is not enabled for the database. Read and write access to data and configuration
is unrestricted
------

test> show dbs
admin   40.00 KiB
config  36.00 KiB
local   64.00 KiB
test> use vehicles
switched to db vehicles
vehicles> show dbs
admin   40.00 KiB
config  72.00 KiB
local   64.00 KiB
vehicles>
```

3. Create a collection called 'two_wheelers'. (use capping) and Create a collection called 'four_wheelers'.

Output:

```
mongosh mongodb://127.0.0.

C:\Users\HIMAVAMSI>mongosh
Current Mongosh Log ID: 69a04845f3137b64f57c2906
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.7.0
Using MongoDB:          8.2.5
Using Mongosh:          2.7.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

------
   The server generated these startup warnings when booting
   2026-02-26T18:08:27.104+05:30: Access control is not enabled for the database. Read and write access to data and configuration
is unrestricted
------

test> use vehicles
switched to db vehicles
vehicles> show dbs
admin    40.00 KiB
config  104.00 KiB
local    64.00 KiB
vehicles> db.createCollection("two_wheelers", {capped: true, size: 50000, max: 100})
{ ok: 1 }
vehicles> db.createCollection("four_wheelers")
{ ok: 1 }
vehicles> show dbs
admin      40.00 KiB
config    104.00 KiB
local      64.00 KiB
vehicles   16.00 KiB
vehicles>
```

4. Add 5 two-wheeler details to the collection named 'two_wheelers'. Each document consists of following fields as bike_name, model (gear or gearless), category (100cc, 125cc, 150cc, 200cc), colors_available (red, black, blue, sport red etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Output:

```
mongosh mongodb://127.0.0.

local      64.00 KiB
vehicles   16.00 KiB
vehicles> db.two_wheelers.insertMany([
| {
| bike_name: "Triumph Speed 400", model: "gear", category: "400cc",
| colors_available: ["red","black","silver"], manufacturer: "Triumph",
| performance: 9, timestamp: new Date("2023-07-05"), price: 233000
| },
| {
| bike_name: "RE Interceptor 650", model: "gear", category: "650cc",
| colors_available: ["orange","black","chrome"], manufacturer: "Royal Enfield",
| performance: 9, timestamp: new Date("2019-11-15"), price: 320000
| },
| {
| bike_name: "RE Hunter 350", model: "gear", category: "350cc",
| colors_available: ["blue","black","red"], manufacturer: "Royal Enfield",
| performance: 8, timestamp: new Date("2022-08-07"), price: 170000
| },
| {
| bike_name: "Apache RR 310", model: "gear", category: "310cc",
| colors_available: ["race red","black"], manufacturer: "TVS",
| performance: 9, timestamp: new Date("2021-09-20"), price: 272000
| },
| {
| bike_name: "Jupiter 125", model: "gearless", category: "125cc",
| colors_available: ["white","grey","blue"], manufacturer: "TVS",
| performance: 8, timestamp: new Date("2022-03-18"), price: 92000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a04908f3137b64f57c2907'),
    '1': ObjectId('69a04908f3137b64f57c2908'),
    '2': ObjectId('69a04908f3137b64f57c2909'),
    '3': ObjectId('69a04908f3137b64f57c290a'),
    '4': ObjectId('69a04908f3137b64f57c290b')
  }
}
vehicles>
```

5. Add 5 four-wheeler details to the collection named ‘four_wheelers’. Each document consists of following fields as vehicle_name, model (commercial or own), category (car, lorry, bus, mini truck, heavy truck, containers), variants (vxi, zxi, petrol, diesel etc) as array, manufacturer, performance (out of 10), timestamp (date and year release) and price.

Output:

```
}
vehicles> db.four_wheelers.insertMany([
| {
| vehicle_name: "Toyota Innova Crysta", model: "commercial", category: "car",variants: ["zxi","diesel"], manufacturer: "Toyota",performance: 9, timestamp: n
ew Date("2021-06-15"), price: 2400000
| },
| {
| vehicle_name: "Mahindra Scorpio N", model: "own", category: "car",
| variants: ["z8","diesel"], manufacturer: "Mahindra",performance: 9, timestamp: new Date("2022-07-10"), price: 2000000
| },
| {
| vehicle_name: "Tata Nexon", model: "own", category: "car",
| variants: ["xms","petrol"], manufacturer: "Tata",performance: 8, timestamp: new Date("2023-01-05"), price: 1200000
| },
| {
| vehicle_name: "Ashok Leyland Dost", model: "commercial", category: "mini truck",
| variants: ["diesel"], manufacturer: "Ashok Leyland",performance: 7, timestamp: new Date("2020-09-18"), price: 750000
| },
| {
| vehicle_name: "Eicher Pro 3015", model: "commercial", category: "heavy truck",
| variants: ["diesel"], manufacturer: "Eicher",performance: 8, timestamp: new Date("2019-11-22"), price: 2800000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a049d5f3137b64f57c290c'),
    '1': ObjectId('69a049d5f3137b64f57c290d'),
    '2': ObjectId('69a049d5f3137b64f57c290e'),
    '3': ObjectId('69a049d5f3137b64f57c290f'),
    '4': ObjectId('69a049d5f3137b64f57c2910')
  }
}
vehicles>
```

6. Write a MongoDB query to display all documents available in two_wheelers and four_wheelers.

Output:

```
mongosh mongodb://127.0.0.

vehicles> db.two_wheelers.find()
[
  {
    _id: ObjectId('69a04908f3137b64f57c2907'),
    bike_name: 'Triumph Speed 400',
    model: 'gear',
    category: '400cc',
    colors_available: [ 'red', 'black', 'silver' ],
    manufacturer: 'Triumph',
    performance: 9,
    timestamp: ISODate('2023-07-05T00:00:00.000Z'),
    price: 233000
  },
  {
    _id: ObjectId('69a04908f3137b64f57c2908'),
    bike_name: 'RE Interceptor 650',
    model: 'gear',
    category: '650cc',
    colors_available: [ 'orange', 'black', 'chrome' ],
    manufacturer: 'Royal Enfield',
    performance: 9,
    timestamp: ISODate('2019-11-15T00:00:00.000Z'),
    price: 320000
  },
  {
    _id: ObjectId('69a04908f3137b64f57c2909'),
    bike_name: 'RE Hunter 350',
    model: 'gear',
    category: '350cc',
    colors_available: [ 'blue', 'black', 'red' ],
    manufacturer: 'Royal Enfield',
    performance: 8,
    timestamp: ISODate('2022-08-07T00:00:00.000Z'),
    price: 170000
  },
  {
    _id: ObjectId('69a04908f3137b64f57c290a'),
    bike_name: 'Apache RR 310',
    model: 'gear',
    category: '310cc',
    colors_available: [ 'race red', 'black' ],
    manufacturer: 'TVS',
    performance: 9,
    timestamp: ISODate('2021-09-20T00:00:00.000Z'),
    price: 272000
  },
  {
    _id: ObjectId('69a04908f3137b64f57c290b'),
    bike_name: 'Jupiter 125',
    model: 'gearless',
    category: '125cc',
    colors_available: [ 'white', 'grey', 'blue' ],
    manufacturer: 'TVS',
    performance: 8,
    timestamp: ISODate('2022-03-18T00:00:00.000Z'),
    price: 92000
  }
```

7. Write a MongoDB query to display only vehicle name and price in all the collection of the database

Output:

```
mongosh mongodb://127.0.0.
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2020-09-18T00:00:00.000Z'),
    price: 750000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c2910'),
    vehicle_name: 'Eicher Pro 3015',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 8,
    timestamp: ISODate('2019-11-22T00:00:00.000Z'),
    price: 2800000
  }
]
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Triumph Speed 400', price: 233000 },
  { bike_name: 'RE Interceptor 650', price: 320000 },
  { bike_name: 'RE Hunter 350', price: 170000 },
  { bike_name: 'Apache RR 310', price: 272000 },
  { bike_name: 'Jupiter 125', price: 92000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Toyota Innova Crysta', price: 2400000 },
  { vehicle_name: 'Mahindra Scorpio N', price: 2000000 },
  { vehicle_name: 'Tata Nexon', price: 1200000 },
  { vehicle_name: 'Ashok Leyland Dost', price: 750000 },
  { vehicle_name: 'Eicher Pro 3015', price: 2800000 }
]
vehicles>
```

8. Write a MongoDB query to display two_wheelers from a particular company

Output:

```
mongosh mongodb://127.0.0.
    price: 2800000
  }
]
vehicles> db.two_wheelers.find({}, {bike_name:1, price:1, _id:0})
[
  { bike_name: 'Triumph Speed 400', price: 233000 },
  { bike_name: 'RE Interceptor 650', price: 320000 },
  { bike_name: 'RE Hunter 350', price: 170000 },
  { bike_name: 'Apache RR 310', price: 272000 },
  { bike_name: 'Jupiter 125', price: 92000 }
]
vehicles> db.four_wheelers.find({}, {vehicle_name:1, price:1, _id:0})
[
  { vehicle_name: 'Toyota Innova Crysta', price: 2400000 },
  { vehicle_name: 'Mahindra Scorpio N', price: 2000000 },
  { vehicle_name: 'Tata Nexon', price: 1200000 },
  { vehicle_name: 'Ashok Leyland Dost', price: 750000 },
  { vehicle_name: 'Eicher Pro 3015', price: 2800000 }
]
vehicles> db.two_wheelers.find({manufacturer:"Triumph"})
[
  {
    _id: ObjectId('69a04908f3137b64f57c2907'),
    bike_name: 'Triumph Speed 400',
    model: 'gear',
    category: '400cc',
    colors_available: [ 'red', 'black', 'silver' ],
    manufacturer: 'Triumph',
    performance: 9,
    timestamp: ISODate('2023-07-05T00:00:00.000Z'),
    price: 233000
  }
]
vehicles> db.two_wheelers.find({manufacturer:"Honda"})

vehicles>
```

9. Write a MongoDB query to display four_wheelers available in diesel variants

Output:

```
mongosh mongodb://127.0.0.
vehicles> db.four_wheelers.find({variants:"diesel"})
[
  {
    _id: ObjectId('69a049d5f3137b64f57c290c'),
    vehicle_name: 'Toyota Innova Crysta',
    model: 'commercial',
    category: 'car',
    variants: [ 'zxi', 'diesel' ],
    manufacturer: 'Toyota',
    performance: 9,
    timestamp: ISODate('2021-06-15T00:00:00.000Z'),
    price: 2400000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c290d'),
    vehicle_name: 'Mahindra Scorpio N',
    model: 'own',
    category: 'car',
    variants: [ 'z8', 'diesel' ],
    manufacturer: 'Mahindra',
    performance: 9,
    timestamp: ISODate('2022-07-10T00:00:00.000Z'),
    price: 2000000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c290f'),
    vehicle_name: 'Ashok Leyland Dost',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2020-09-18T00:00:00.000Z'),
    price: 750000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c2910'),
    vehicle_name: 'Eicher Pro 3015',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 8,
    timestamp: ISODate('2019-11-22T00:00:00.000Z'),
    price: 2800000
  }
]
```

10. Write a MongoDB query to display vehicles name, category and manufacturer details whose rating is more than 5.

Output:

```
mongosh mongodb://127.0.0.

    timestamp: ISODate('2022-07-10T00:00:00.000Z'),
    price: 2000000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c290f'),
    vehicle_name: 'Ashok Leyland Dost',
    model: 'commercial',
    category: 'mini truck',
    variants: [ 'diesel' ],
    manufacturer: 'Ashok Leyland',
    performance: 7,
    timestamp: ISODate('2020-09-18T00:00:00.000Z'),
    price: 750000
  },
  {
    _id: ObjectId('69a049d5f3137b64f57c2910'),
    vehicle_name: 'Eicher Pro 3015',
    model: 'commercial',
    category: 'heavy truck',
    variants: [ 'diesel' ],
    manufacturer: 'Eicher',
    performance: 8,
    timestamp: ISODate('2019-11-22T00:00:00.000Z'),
    price: 2800000
  }
]
vehicles> db.two_wheelers.find({performance:{$gt:5}}, {bike_name:1, category:1, manufacturer:1, _id:0})
[
  {
    bike_name: 'Triumph Speed 400',
    category: '400cc',
    manufacturer: 'Triumph'
  },
  {
    bike_name: 'RE Interceptor 650',
    category: '650cc',
    manufacturer: 'Royal Enfield'
  },
  {
    bike_name: 'RE Hunter 350',
    category: '350cc',
    manufacturer: 'Royal Enfield'
  },
  { bike_name: 'Apache RR 310', category: '310cc', manufacturer: 'TVS' },
  { bike_name: 'Jupiter 125', category: '125cc', manufacturer: 'TVS' }
]
vehicles>
```

2. Use MongoDB to implement the following DB operations for a Zoo

1. Create a database called 'animal' and *write* a MongoDB query to select database as'animal'.

Output:

```
  },
  {
    bike_name: 'RE Interceptor 650',
    category: '650cc',
    manufacturer: 'Royal Enfield'
  },
  {
    bike_name: 'RE Hunter 350',
    category: '350cc',
    manufacturer: 'Royal Enfield'
  },
  { bike_name: 'Apache RR 310', category: '310cc', manufacturer: 'TVS' },
  { bike_name: 'Jupiter 125', category: '125cc', manufacturer: 'TVS' }
]
vehicles> use animal
switched to db animal
animal>
```

2. Write a MongoDB query to display all the databases.

Output:

```
  },
  {
    bike_name: 'RE Interceptor 650',
    category: '650cc',
    manufacturer: 'Royal Enfield'
  },
  {
    bike_name: 'RE Hunter 350',
    category: '350cc',
    manufacturer: 'Royal Enfield'
  },
  { bike_name: 'Apache RR 310', category: '310cc', manufacturer: 'TVS' },
  { bike_name: 'Jupiter 125', category: '125cc', manufacturer: 'TVS' }
]
vehicles> use animal
switched to db animal
animal> show dbs
admin     40.00 KiB
config    96.00 KiB
local     64.00 KiB
vehicles  80.00 KiB
animal>
```

3. Create a collection called 'wild_animals'.(use capping) and Create a collection called 'domestic_animals'.

Output:

```
mongosh mongodb://127.0.0.
  },
  {
    bike_name: 'RE Interceptor 650',
    category: '650cc',
    manufacturer: 'Royal Enfield'
  },
  {
    bike_name: 'RE Hunter 350',
    category: '350cc',
    manufacturer: 'Royal Enfield'
  },
  { bike_name: 'Apache RR 310', category: '310cc', manufacturer: 'TVS' },
  { bike_name: 'Jupiter 125', category: '125cc', manufacturer: 'TVS' }
]
vehicles> use animal
switched to db animal
animal> show dbs
admin     40.00 KiB
config    96.00 KiB
local     64.00 KiB
vehicles  80.00 KiB
animal> db.createCollection("wild_animals", {
|    capped:true,
|    size:50000,
|    max:100
| })
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal>
```

4. Add 5 wild_animal details to the collection named 'wild_animals'. Each document consists of following fields as animal_name, nature (harm or harmless), favorite_foods (meat, rabbits, deer etc) as array, care_taker_name, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Output:

```
mongosh mongodb://127.0.0.
{ ok: 1 }
animal> db.createCollection("domestic_animals")
{ ok: 1 }
animal> db.wild_animals.insertMany([
| {
| animal_name: "Lion", nature: "harm",favorite_foods: ["meat","deer"], care_taker_name: "Ravi",
| life_span: 14, timestamp: new Date("2018-03-12"), expenses: 50000
| },
| {
| animal_name: "Tiger", nature: "harm",favorite_foods: ["meat"], care_taker_name: "Arjun",
| life_span: 15, timestamp: new Date("2019-07-25"), expenses: 60000
| },
| {
| animal_name: "Elephant", nature: "harmless",favorite_foods: ["grass","fruits"], care_taker_name: "Suresh",
| life_span: 60, timestamp: new Date("2016-11-08"), expenses: 70000
| },
| {
| animal_name: "Bear", nature: "harm",favorite_foods: ["fish","honey"], care_taker_name: "Ravi",
| life_span: 25, timestamp: new Date("2020-01-17"), expenses: 45000
| },
| {
| animal_name: "Deer", nature: "harmless",favorite_foods: ["grass"], care_taker_name: "Kiran",
| life_span: 18, timestamp: new Date("2021-09-03"), expenses: 20000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a04d72f3137b64f57c2911'),
    '1': ObjectId('69a04d72f3137b64f57c2912'),
    '2': ObjectId('69a04d72f3137b64f57c2913'),
    '3': ObjectId('69a04d72f3137b64f57c2914'),
    '4': ObjectId('69a04d72f3137b64f57c2915')
  }
}
animal>
```

5. Add 5 domestic-animal details to the collection named 'domestic_animals'. Each document consists of following fields as animal_name, gender (male or female), favorite_foods (meat, rabbits, deer etc) as array, animal_petname, life span (in years), timestamp (when the animal registered at the Zoo) and expenses.

Output:

```
animal> db.domestic_animals.insertMany([
| {
| animal_name: "Dog", gender: "male",favorite_foods: ["meat","rice"], animal_petname: "Rocky",life_span: 12,
| timestamp: new Date("2022-02-14"), expenses: 10000
| },
| {
| animal_name: "Cat", gender: "female",favorite_foods: ["fish","milk"], animal_petname: "Misty",life_span: 14,
| timestamp: new Date("2021-05-09"), expenses: 8000
| },
| {
| animal_name: "Cow", gender: "female",favorite_foods: ["grass"], animal_petname: "Gauri",life_span: 20,
|  timestamp: new Date("2017-12-21"), expenses: 15000
| },
| {
| animal_name: "Goat", gender: "male",favorite_foods: ["leaves"], animal_petname: "Chotu",life_span: 10,
| timestamp: new Date("2023-04-11"), expenses: 6000
| },
| {
| animal_name: "Rabbit", gender: "female",favorite_foods: ["carrot"], animal_petname: "Bunny",life_span: 8,
| timestamp: new Date("2020-08-30"), expenses: 4000
| }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69a04de1f3137b64f57c2916'),
    '1': ObjectId('69a04de1f3137b64f57c2917'),
    '2': ObjectId('69a04de1f3137b64f57c2918'),
    '3': ObjectId('69a04de1f3137b64f57c2919'),
    '4': ObjectId('69a04de1f3137b64f57c291a')
  }
}
animal>
```

6. Write a MongoDB query to display all documents available in wild_animals and domastic_animals.

Output:

```
mongosh mongodb://127.0.0.

animal> db.wild_animals.find()
[
  {
    _id: ObjectId('69a04d72f3137b64f57c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2018-03-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69a04d72f3137b64f57c2912'),
    animal_name: 'Tiger',
    nature: 'harm',
    favorite_foods: [ 'meat' ],
    care_taker_name: 'Arjun',
    life_span: 15,
    timestamp: ISODate('2019-07-25T00:00:00.000Z'),
    expenses: 60000
  },
  {
    _id: ObjectId('69a04d72f3137b64f57c2913'),
    animal_name: 'Elephant',
    nature: 'harmless',
    favorite_foods: [ 'grass', 'fruits' ],
    care_taker_name: 'Suresh',
    life_span: 60,
    timestamp: ISODate('2016-11-08T00:00:00.000Z'),
    expenses: 70000
  },
  {
    _id: ObjectId('69a04d72f3137b64f57c2914'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Ravi',
    life_span: 25,
    timestamp: ISODate('2020-01-17T00:00:00.000Z'),
    expenses: 45000
  },
  {
    _id: ObjectId('69a04d72f3137b64f57c2915'),
    animal_name: 'Deer',
    nature: 'harmless',
    favorite_foods: [ 'grass' ],
    care_taker_name: 'Kiran',
    life_span: 18,
    timestamp: ISODate('2021-09-03T00:00:00.000Z'),
    expenses: 20000
  }
]
```

7. Write a MongoDB query to display only animal name and expenses in all the collection of the database

Output:

```
mongosh mongodb://127.0.0.

    life_span: 20,
    timestamp: ISODate('2017-12-21T00:00:00.000Z'),
    expenses: 15000
  },
  {
    _id: ObjectId('69a04de1f3137b64f57c2919'),
    animal_name: 'Goat',
    gender: 'male',
    favorite_foods: [ 'leaves' ],
    animal_petname: 'Chotu',
    life_span: 10,
    timestamp: ISODate('2023-04-11T00:00:00.000Z'),
    expenses: 6000
  },
  {
    _id: ObjectId('69a04de1f3137b64f57c291a'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrot' ],
    animal_petname: 'Bunny',
    life_span: 8,
    timestamp: ISODate('2020-08-30T00:00:00.000Z'),
    expenses: 4000
  }
]
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Bear', expenses: 45000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 10000 },
  { animal_name: 'Cat', expenses: 8000 },
  { animal_name: 'Cow', expenses: 15000 },
  { animal_name: 'Goat', expenses: 6000 },
  { animal_name: 'Rabbit', expenses: 4000 }
]
animal>
```

8. Write a MongoDB query to display domestic_animals whose life is a particular year

Output:

```
mongosh mongodb://127.0.0.
  },
  {
    _id: ObjectId('69a04de1f3137b64f57c291a'),
    animal_name: 'Rabbit',
    gender: 'female',
    favorite_foods: [ 'carrot' ],
    animal_petname: 'Bunny',
    life_span: 8,
    timestamp: ISODate('2020-08-30T00:00:00.000Z'),
    expenses: 4000
  }
]
animal> db.wild_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Lion', expenses: 50000 },
  { animal_name: 'Tiger', expenses: 60000 },
  { animal_name: 'Elephant', expenses: 70000 },
  { animal_name: 'Bear', expenses: 45000 },
  { animal_name: 'Deer', expenses: 20000 }
]
animal> db.domestic_animals.find({}, {animal_name:1, expenses:1, _id:0})
[
  { animal_name: 'Dog', expenses: 10000 },
  { animal_name: 'Cat', expenses: 8000 },
  { animal_name: 'Cow', expenses: 15000 },
  { animal_name: 'Goat', expenses: 6000 },
  { animal_name: 'Rabbit', expenses: 4000 }
]
animal> db.domestic_animals.find({life_span:12})
[
  {
    _id: ObjectId('69a04de1f3137b64f57c2916'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'rice' ],
    animal_petname: 'Rocky',
    life_span: 12,
    timestamp: ISODate('2022-02-14T00:00:00.000Z'),
    expenses: 10000
  }
]
animal> |
```

9. Write a MongoDB query to display wild_animals available under a particular care_taker

Output:

```
mongosh mongodb://127.0.0.
  { animal_name: 'Cat', expenses: 8000 },
  { animal_name: 'Cow', expenses: 15000 },
  { animal_name: 'Goat', expenses: 6000 },
  { animal_name: 'Rabbit', expenses: 4000 }
]
animal> db.domestic_animals.find({life_span:12})
[
  {
    _id: ObjectId('69a04de1f3137b64f57c2916'),
    animal_name: 'Dog',
    gender: 'male',
    favorite_foods: [ 'meat', 'rice' ],
    animal_petname: 'Rocky',
    life_span: 12,
    timestamp: ISODate('2022-02-14T00:00:00.000Z'),
    expenses: 10000
  }
]
animal> db.wild_animals.find({care_taker_name:"Ravi"})
[
  {
    _id: ObjectId('69a04d72f3137b64f57c2911'),
    animal_name: 'Lion',
    nature: 'harm',
    favorite_foods: [ 'meat', 'deer' ],
    care_taker_name: 'Ravi',
    life_span: 14,
    timestamp: ISODate('2018-03-12T00:00:00.000Z'),
    expenses: 50000
  },
  {
    _id: ObjectId('69a04d72f3137b64f57c2914'),
    animal_name: 'Bear',
    nature: 'harm',
    favorite_foods: [ 'fish', 'honey' ],
    care_taker_name: 'Ravi',
    life_span: 25,
    timestamp: ISODate('2020-01-17T00:00:00.000Z'),
    expenses: 45000
  }
]
animal> |
```

10. Write a MongoDB query to display animal name, favorite_foods and expenses details whose lifespan is more than 5 years.

Output:

```
mongosh mongodb://127.0.0.

animal> db.wild_animals.find(
|  {life_span:{$gt:5}},
|  {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  { animal_name: 'Tiger', favorite_foods: [ 'meat' ], expenses: 60000 },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits' ],
    expenses: 70000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'honey' ],
    expenses: 45000
  },
  { animal_name: 'Deer', favorite_foods: [ 'grass' ], expenses: 20000 }
]
animal> db.wild_animals.find(
|  {life_span:{$gt:5}},
|  {animal_name:1, favorite_foods:1, expenses:1, _id:0}
| )
[
  {
    animal_name: 'Lion',
    favorite_foods: [ 'meat', 'deer' ],
    expenses: 50000
  },
  { animal_name: 'Tiger', favorite_foods: [ 'meat' ], expenses: 60000 },
  {
    animal_name: 'Elephant',
    favorite_foods: [ 'grass', 'fruits' ],
    expenses: 70000
  },
  {
    animal_name: 'Bear',
    favorite_foods: [ 'fish', 'honey' ],
    expenses: 45000
  },
  { animal_name: 'Deer', favorite_foods: [ 'grass' ], expenses: 20000 }
]
animal>
```